

DSCEngine Security Audit Report

Multi-Collateral Stability Engine

May 10, 2026

Contents

About CableGraph Audit Group	1
Disclaimer	1
Risk Classification	1
Protocol Summary	2
Audit Scope	2
Executive Summary	3
Findings	7
Critical Risk	7
[C-01] Ownable constructor never initializes owner — protocol permanently DOA	7
[C-02] emergencyWithdraw drains all collateral with zero accounting update	9
High Risk	11
[H-01] _calculateHealthFactor arguments swapped — all HF values inverted	11
[H-02] depositCollateralAndMintDSC permanently broken — double non-Reentrant	13
[H-03] redeemCollateral and burnDSC missing notPaused modifier	14
[H-04] Liquidation collateral sent before DSC pulled — token flow ordering risk	16
Medium Risk	17
[M-01] LIQUIDATION_THRESHOLD = 150 — comment falsely claims 200%	17
[M-02] Oracle: missing price > 0 and answeredInRound checks	17
[M-03] ERC20YulLib — no return value validation on low-level calls	18
Low Risk	19
[L-01] governance address declared public but never written or used	19
[L-02] Duplicate view functions with inconsistent naming	19
Informational	20
[I-01] Invariant Handler: oracle price never updated — critical coverage gap	20

Test Suite Analysis	21
Post-Audit Recommendations	22
Immediate — before any deployment	22
Before mainnet launch	22
Architecture recommendations	23
On the mitigation review	23



DSCEngine Security Audit Report

CableGraph Audit Group

Conducted by: **Dennis Kiptoo**

May 5, 2026 — May 10, 2026

About CableGraph Audit Group

CableGraph Audit Group is an independent smart contract security research practice specialising in DeFi protocol security, stablecoin systems, oracle integrations, and ERC20 token mechanics. Audit methodology combines structured multi-pass manual review, static analysis with Slither, symbolic execution with Mythril, and property-based fuzzing with Foundry invariant testing.

Lead Researcher: Dennis Kiptoo

Portfolio: <https://github.com/cableGraph>

Contact: Open to audit engagements and research collaborations.

Disclaimer

CableGraph Audit Group makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts. This report does not constitute financial or legal advice.

Risk Classification

	High Impact	Medium Impact	Low Impact
High Likelihood	Critical	High	Medium
Medium Likelihood	High	Medium	Low
Low Likelihood	Medium	Low	Low

Protocol Summary

DSCEngine is a minimalistic decentralized stablecoin protocol where users lock exogenous crypto collateral (wETH and wBTC) to mint DSC — a USD-pegged algorithmic stablecoin. The system is modelled closely after MakerDAO's DAI, with no governance fees and a purely algorithmic supply control mechanism.

Design requirements:

- System must always be overcollateralized: total collateral value $\geq$ total DSC supply at all times
- No user's health factor should drop below $1e18$ without being liquidatable
- Liquidations must always improve a user's health factor
- Emergency mechanisms must not corrupt protocol accounting

Collateral tokens: wETH (18 decimals), wBTC (8 decimals)

Oracle: Chainlink price feeds via `OracleLib` stale-check wrapper

Math layer: `EngineMath` library — WAD arithmetic, health factor, USD value

Audit Scope

Item	Detail
Repository	DSCEngine — Multi-Collateral Stability Engine
Commit	[e7231d9e86d18e4704f88b86cf84c01da9b2bbd6]
Fix Commit	Pending mitigation review
Audit Timeline	May 5 — May 10, 2026
Methods	Manual Review (5-pass), Slither, Mythril, Foundry Fuzz + Invariant
Solidity Version	$\sim 0.8.18$

Files in scope:

`src/Core/DSCEngine.sol`

```
src/Core/DecentralizedStableCoin.sol
src/Libraries/EngineMath.sol
src/Libraries/OracleLib.sol
src/Libraries/ERC20YulLib.sol
src/Libraries/AccountDataPackerLib.sol
```

Executive Summary

Over 5 days, a full-scope security audit was conducted on the DSCEngine protocol. The audit applied a structured 5-pass manual review system — trust model, arithmetic, external calls, composability, and end-to-end liquidation walkthrough — followed by automated tooling and a complete Foundry test suite written from scratch.

A total of 12 issues were found.

Severity	Count	Status
Critical	2	Open
High	4	Open
Medium	3	Open
Low	2	Open
Informational	1	Open
Total	12	

Summary of Findings:

ID	Severity	Title	Status
[C-01]	Critical	Ownable constructor never initializes owner — protocol permanently DOA	Open

ID	Severity	Title	Status
[C-02]	Critical	emergencyWithdraw drains all collateral with zero accounting update	Open
[H-01]	High	_calculateHealthFactor arguments swapped — all HF values inverted	Open
[H-02]	High	depositCollateralAndMintDSC permanently broken — double nonReentrant	Open
[H-03]	High	redeemCollateral and burnDSC missing notPaused modifier	Open
[H-04]	High	Liquidation collateral sent before DSC pulled — ordering risk	Open
[M-01]	Medium	LIQUIDATION_THRESHOLD = 150 — comment falsely claims 200%	Open
[M-02]	Medium	Oracle: missing price > 0 and answeredInRound checks	Open
[M-03]	Medium	ERC20YulLib — no return value validation on low-level calls	Open

ID	Severity	Title	Status
[L-01]	Low	governance address declared public but never written or used	Open
[L-02]	Low	Duplicate view functions with inconsistent naming	Open
[I-01]	Info	Invariant Handler: price never updated — critical coverage gap	Open

Key observations:

The two Critical findings are independent but compounding. C-01 renders the DSC token permanently unmintable because the `Ownable` constructor is called incorrectly — the entire protocol is non-functional from deployment. C-02 is an architectural flaw that allows the owner to drain 100% of user collateral with a single transaction and no accounting update.

Of the four High findings, H-01 is the most impactful: every health factor calculation in the protocol receives its arguments in the wrong order, producing values that are the mathematical inverse of the intended result. Every healthy user appears liquidatable; every insolvent user appears safe.

Test Suite Analysis:

The protocol had an existing Foundry fuzz suite covering the happy path. Via the 5-pass methodology, it was possible to identify critical vulnerabilities that the test suite did not detect — including C-01 (`Ownable` construction), C-02 (emergency drain), H-01 (argument swap), and H-02 (nonReentrant deadlock). A targeted test suite was written from scratch as an additional deliverable, covering all Critical and High findings with working Foundry PoC tests.

The invariant handler was found to have a significant coverage gap: oracle prices never change during fuzzing, meaning the overcollateralization invariant never encounters a price crash scenario. An expanded handler with `updateCollateralPrice()`, `triggerLiquidation()`, and `burnDSC()` was contributed to the protocol.

Findings

Critical Risk

[C-01] Ownable constructor never initializes owner — protocol permanently DOA

Severity: Critical

Likelihood: High | **Impact:** High

Affected Contract: `src/Core/DecentralizedStableCoin.sol :: constructor()`

Description:

The `DecentralizedStableCoin` constructor calls `Ownable(msg.sender)` as a standalone expression inside the function body. In OpenZeppelin v5, `Ownable` is an abstract contract whose constructor must be invoked in the inheritance list — a body-level call is a syntactic no-op that executes nothing.

```
// Current - BROKEN
constructor() ERC20("Decentralized Stable Coin", "DSC") {
    Ownable(msg.sender); // ← body expression, not base constructor call
}
```

As a result, `owner()` returns `address(0)`. The `onlyOwner` modifier on both `mint()` and `burn()` will always revert for any caller, including the `DSCEngine` contract that is meant to govern the token. The DSC token can never be minted. The protocol is non-functional from block zero.

Impact:

- `DSCEngine.mintDSC()` calls `i_dsc.mint()` — always reverts with `OwnableUnauthorizedAccount`
- `DSCEngine.liquidate()` calls `i_dsc.burn()` — always reverts
- The `transferOwnership()` path is also broken: only the current owner (`address(0)`) can transfer — reverting for every real caller

- The entire protocol is permanently non-functional with no upgrade path

Proof of Concept:

```
forge test --match-test test_C01_OwnableConstructorNeverSetsOwner -vvvv
# Result: PASS - owner() == address(0) confirmed
```

```
function test_C01_OwnableConstructorNeverSetsOwner() public view {
    assertEq(dsc.owner(), address(0));
    assertTrue(dsc.owner() != DEPLOYER);
}

function test_C01_MintRevertsForEveryCallerBecauseOwnerIsZero() public {
    vm.prank(DEPLOYER);
    vm.expectRevert(); // OwnableUnauthorizedAccount
    dsc.mint(DEPLOYER, 1e18);

    vm.prank(ANY_USER);
    vm.expectRevert();
    dsc.mint(ANY_USER, 1e18);
}
```

Recommended Fix:

Move the Ownable initialization into the inheritance list, not the constructor body:

```
// Fixed
constructor()
    ERC20("Decentralized Stable Coin", "DSC")
    Ownable(msg.sender) // ← inheritance list invocation
{}
```

Client Response: Awaiting.

Mitigation Status: Unresolved — pending mitigation review.

[C-02] emergencyWithdraw drains all collateral with zero accounting update**Severity:** Critical**Likelihood:** High | **Impact:** High**Affected Contract:** `src/Core/DSCEngine.sol :: emergencyWithdraw(address token)`**Description:**

The `emergencyWithdraw` function transfers 100% of a token's balance from the contract to the owner with no updates to `s_accounts[user].collateral` mappings and no corresponding DSC burn. Any user whose collateral is drained retains their full DSC debt balance with zero backing collateral. The function also accepts any token address — including the DSC token itself — with no validation.

```
function emergencyWithdraw(address token) external {
    require(msg.sender == s_protocolState.owner, "Not owner");
    require(s_protocolState.paused, "Not paused");
    // @audit No accounting update - user.collateral mapping unchanged
    // @audit No DSC burn - unbacked debt remains
    // @audit No token validation - accepts ANY address
    uint256 balance = token.balanceOf(address(this));
    token.safeTransfer(s_protocolState.owner, balance);
}
```

The owner is a single EOA with no timelock and no multisig requirement.

Impact:

- Owner drains 100% of all user collateral in a single transaction
- All affected users retain DSC debt with zero collateral backing
- The solvency invariant `wethValue + wbtcValue >= totalSupply` is permanently broken
- No on-chain recourse for affected users
- Classic rug-pull vector — requires only owner key compromise or insider action

Proof of Concept:

```
forge test --match-test test_C02_EmergencyWithdrawDrainsWithoutAccountingUpdate -vvvv
# Result: PASS - contract balance = 0, user accounting unchanged
```

```
function test_C02_EmergencyWithdrawDrainsWithoutAccountingUpdate() public {
    vm.startPrank(USER);
    ERC20Mock(weth).approve(address(dscE), AMOUNT_COLLATERAL);
    dscE.depositCollateral(weth, AMOUNT_COLLATERAL);
    dscE.mintDSC(5000e18);
    vm.stopPrank();

    dscE.pause();
    dscE.emergencyWithdraw(weth);

    assertEq(ERC20Mock(weth).balanceOf(address(dscE)), 0);
    assertEq(dscE.getCollateralBalanceOfUser(USER, weth), AMOUNT_COLLATERAL);
    assertEq(dsc.totalSupply(), 5000e18);
    assertTrue(dscE.getUsdValue(weth, 0) < dsc.totalSupply());
}
```

Recommended Fix:

Option 1 (recommended): Remove `emergencyWithdraw` entirely. A truly decentralized protocol should not have a unilateral owner drain mechanism.

Option 2 (if emergency mechanism is required): Replace with a time-locked withdrawal (minimum 72-hour delay) requiring a multi-sig (3-of-5), per-user accounting update, proportional DSC burn, and granular event emission.

```
// Minimum validation to add:
require(s_tokenConfigs[token].isActive, "Not a collateral token");
// Update user accounting before transfer (CEI)
// Burn corresponding DSC proportionally
```

Client Response: Awaiting.

Mitigation Status: Unresolved — pending mitigation review.

High Risk

[H-01] `_calculateHealthFactor` arguments swapped — all HF values inverted

Severity: High

Likelihood: High | **Impact:** High

Affected Contract: `src/Core/DSCEngine.sol :: _calculateHealthFactor()` and all call sites

Description:

The internal function `_calculateHealthFactor` has the signature:

```
function _calculateHealthFactor(  
    uint256 totalDSCMinted,      // parameter 1  
    uint256 collateralValueInUsd // parameter 2  
) internal pure returns (uint256)
```

Every call site in the contract passes arguments in the **reverse order** — (`collateralValueInUsd`, `totalDSCMinted`). The `EngineMath.calculateHealthFactor` library function receives them inverted, producing a health factor that is the mathematical reciprocal of the correct value.

```
// _healthFactor() call site - arguments SWAPPED:  
return _calculateHealthFactor(  
    collateralValueInUsd, // ← passed as totalDSCMinted (WRONG)  
    totalDSCMinted       // ← passed as collateralValueInUsd (WRONG)  
);
```

Mathematical proof:

Correct: $HF = (\text{collateral} \times 1.5e18) / \text{debt}$
 $= (20000e18 \times 1.5) / 5000e18 = 6.0e18 \rightarrow \text{HEALTHY}$

Swapped: $HF = (\text{debt} \times 1.5e18) / \text{collateral}$

$$= (5000e18 \times 1.5) / 20000e18 = 0.375e18 \rightarrow \text{LIQUIDATABLE}$$

A user with \$20,000 collateral and \$5,000 DSC debt — 4× overcollateralized — appears permanently below the liquidation threshold.

Impact:

- Every healthy user appears liquidatable to the protocol
- Every insolvent user appears safe
- `mintDSC` may allow or block minting based on inverted logic
- Health factor comparisons against `MIN_HEALTH_FACTOR = 1e18` are entirely unreliable
- The core safety guarantee of the protocol does not function

Proof of Concept:

```
forge test --match-test test_H01_ArgumentSwapProducesInvertedHealthFactor -vvvv
# Result: FAIL - actual HF = 0.375e18, expected 6e18
# A FAILING test IS the PoC for this finding
```

```
function test_H01_ArgumentSwapProducesInvertedHealthFactor() public view {
    uint256 actualHF = dscE.calculateHealthFactor(5000e18, 20000e18);
    // Correct: 6e18 | Swapped: 0.375e18
    assertEq(actualHF, 6e18,
        "H-01: Expected 6e18 - if 0.375e18, args swapped");
}
```

Recommended Fix:

```
// Fix - correct argument order:
return _calculateHealthFactor(
    totalDSCMinted,      // ← param 1: DSC minted
    collateralValueInUsd // ← param 2: collateral value
);

// Rename to prevent recurrence:
function _calculateHealthFactor(
    uint256 dscMinted,
    uint256 collateralUsd
```

```
) internal pure returns (uint256) { ... }
```

Client Response: Awaiting.

Mitigation Status: Unresolved — pending mitigation review.

[H-02] depositCollateralAndMintDSC permanently broken — double nonReentrant

Severity: High

Likelihood: High | **Impact:** High

Affected Contract: `src/Core/DSCEngine.sol :: depositCollateralAndMintDSC()`

Description:

`depositCollateralAndMintDSC` carries its own `nonReentrant` modifier and internally calls `depositCollateral` (also `nonReentrant`) and `mintDSC` (also `nonReentrant`). OpenZeppelin's `ReentrancyGuard` uses a `uint256` mutex. When the outer function acquires the lock (`_status = ENTERED`), all inner calls find the mutex locked and revert with `ReentrancyGuardReentrantCall`.

```
function depositCollateralAndMintDSC(...)
    external
    nonReentrant // ← Step 1: mutex locked
    ...
{
    depositCollateral(...); // ← Step 2: nonReentrant → REVERT always
    mintDSC(...);
}
```

Impact:

- Primary user-facing entry point is permanently non-functional
- Every call pays gas for a guaranteed revert
- Users must make two separate transactions — doubling gas and eliminating atomicity
- This function has never successfully executed since deployment

Proof of Concept:


```
forge test --match-test test_H02_DepositCollateralAndMintDSCAlwaysReverts -vvvv
# Result: PASS - guaranteed revert confirmed
```

```
function test_H02_DepositCollateralAndMintDSCAlwaysReverts() public {
    vm.startPrank(USER);
    ERC20Mock(weth).approve(address(dscE), AMOUNT_COLLATERAL);
    vm.expectRevert(); // ReentrancyGuardReentrantCall
    dscE.depositCollateralAndMintDSC(weth, AMOUNT_COLLATERAL, 1000e18);
    vm.stopPrank();

    assertEq(dscE.getCollateralBalanceOfUser(USER, weth), 0);
    assertEq(dsc.balanceOf(USER), 0);
}
```

Recommended Fix:

Remove nonReentrant from the wrapper — inner functions already protect:

```
function depositCollateralAndMintDSC(
    address tokenCollateralAddress,
    uint256 collateralAmount,
    uint256 amountDSCToMint
) external notPaused { // nonReentrant REMOVED
    depositCollateral(tokenCollateralAddress, collateralAmount);
    mintDSC(amountDSCToMint);
}
```

Client Response: Awaiting.

Mitigation Status: Unresolved — pending mitigation review.

[H-03] redeemCollateral and burnDSC missing notPaused modifier

Severity: High

Likelihood: Medium | **Impact:** High

Affected Contract: src/Core/DSCEngine.sol :: redeemCollateral(), burnDSC()

Description:

depositCollateral, mintDSC, and liquidate all carry notPaused. However, redeemCollateral and burnDSC do not. During a pause, users can still exit their positions — directly undermining the pause mechanism. A race condition exists between user redemptions and emergencyWithdraw, where both can drain the same funds simultaneously, corrupting accounting.

Proof of Concept:

```
forge test --match-test test_H03_RedeemCollateralSucceedsDuringPause -vvvv
# Result: PASS - redeemCollateral succeeds while paused
```

```
function test_H03_RedeemCollateralSucceedsDuringPause() public {
    vm.startPrank(USER);
    ERC20Mock(weth).approve(address(dscE), AMOUNT_COLLATERAL);
    dscE.depositCollateral(weth, AMOUNT_COLLATERAL);
    vm.stopPrank();

    dscE.pause();
    vm.prank(USER);
    dscE.redeemCollateral(weth, 1 ether); // SUCCEEDS - bug confirmed

    assertEq(
        dscE.getCollateralBalanceOfUser(USER, weth),
        AMOUNT_COLLATERAL - 1 ether
    );
}
```

Recommended Fix:

```
function redeemCollateral(address tokenCollateralAddress, uint256 amountCollateral)
    public nonReentrant moreThanZero(amountCollateral) notPaused { ... }

function burnDSC(uint256 amount)
    public nonReentrant moreThanZero(amount) notPaused { ... }
```

Client Response: Awaiting.

Mitigation Status: Unresolved — pending mitigation review.

[H-04] Liquidation collateral sent before DSC pulled — token flow ordering risk

Severity: High

Likelihood: Low | **Impact:** High

Affected Contract: src/Core/DSCEngine.sol :: liquidate()

Description:

In liquidate(), collateral is transferred to the liquidator before DSC is pulled from them:

```
// Dangerous ordering:
collateral.safeTransfer(msg.sender, totalCollateralToRedeem);      // OUT first
address(i_dsc).safeTransferFrom(msg.sender, address(this), debtToCover); // IN second
i_dsc.burn(debtToCover);
```

EVM atomicity currently protects against exploitation with WETH/WBTC. However, any non-standard token or future callback mechanism breaks this assumption.

Recommended Fix:

Pull DSC before sending collateral:

```
// Fixed order:
address(i_dsc).safeTransferFrom(msg.sender, address(this), debtToCover); // Pull first
i_dsc.burn(debtToCover);
collateral.safeTransfer(msg.sender, totalCollateralToRedeem);      // Send after
```

Client Response: Awaiting.

Mitigation Status: Unresolved — pending mitigation review.

Medium Risk

[M-01] LIQUIDATION_THRESHOLD = 150 — **comment falsely claims 200%**

Severity: Medium

Likelihood: High | **Impact:** Low

Affected Contract: src/Core/DSCEngine.sol

Description:

```
uint256 private constant LIQUIDATION_THRESHOLD = 150; //200% ← WRONG
```

The constant enforces 150% collateralization (users can mint up to 66.7% of collateral value). The comment claims 200% (50% max mint). These are materially different risk profiles — 150% is significantly more permissive.

Proof of Concept:

```
// Mint above 200% limit - SUCCEEDS, proving 150% enforced not 200%
uint256 aboveDoubleLimit = (collateralUsd / 2) + 1e18;
dscE.mintDSC(aboveDoubleLimit); // Does NOT revert
```

Recommended Fix: uint256 private constant LIQUIDATION_THRESHOLD = 150; // 150% collateralization ratio

[M-02] Oracle: missing price > 0 and answeredInRound checks

Severity: Medium

Likelihood: Low | **Impact:** High

Affected Contract: src/Libraries/OracleLib.sol

Description:

OracleLib checks stale timestamps but not: `price > 0`, `answeredInRound >= roundId`, or L2 sequencer uptime. A Chainlink feed returning `price = 0` during an outage causes every user's collateral value to become 0 simultaneously — triggering mass liquidations with no market basis.

Recommended Fix:

```
require(price > 0, "OracleLib: invalid price");
require(answeredInRound >= roundId, "OracleLib: incomplete round");
require(block.timestamp - updatedAt <= TIMEOUT, "OracleLib: stale price");
```

[M-03] ERC20YulLib — no return value validation on low-level calls**Severity:** Medium**Likelihood:** Low | **Impact:** High**Affected Contract:** `src/Libraries/ERC20YulLib.sol`**Description:**

ERC20YulLib implements ERC20 transfers via inline Yul assembly. If the `CALL` opcode return value is not validated, non-reverting failures (e.g. USDT returning `false`) silently pass — allowing `depositCollateral` to record phantom collateral with no actual token transfer.

Recommended Fix: Replace ERC20YulLib with OpenZeppelin SafeERC20.

Low Risk

[L-01] governance address declared public but never written or used

Severity: Low | **Affected Contract:** `src/Core/DSCEngine.sol`

`address public governance` is declared, never assigned, never read. The commented-out `setGovernanceToken` confirms this is unfinished work. A latent public state slot is an uncontrolled privilege escalation surface if a setter is added later without proper guards.

Recommended Fix: Remove or implement with proper access controls and a time-lock.

[L-02] Duplicate view functions with inconsistent naming

Severity: Low | **Affected Contract:** `src/Core/DSCEngine.sol`

`getCollateralBalanceOfUser(user, token)` and `getCollateralDeposited(user, token)` are identical implementations. Duplicates inflate the ABI surface, confuse integrators, and risk divergence in future refactors.

Recommended Fix: Remove one function and update all references.

Informational

[I-01] Invariant Handler: oracle price never updated — critical coverage gap

The existing handler (`Handler.t.sol`) never calls `MockV3Aggregator.updateAnswer()`. The fuzzer runs in a world where ETH is always \$2,000 — the solvency invariant never encounters a price crash. An expanded handler was contributed to the protocol as an additional deliverable, adding:

- `updateCollateralPrice(uint256 newPrice)` — bounded price crash simulation (\$100-\$4,000)
- `triggerLiquidation(uint256 userSeed, uint256 debtSeed)` — stateful H-01 verification
- `burnDSC(uint256 amount, uint256 userSeed)` — full deposit→mint→burn→redeem lifecycle
- `invariant_noUserWithDSCCanHaveBrokenHealthFactor` — directly catches H-01 in stateful context

Test Suite Analysis

A complete Foundry test suite was written from scratch as an additional audit deliverable, covering all Critical and High findings with working PoC tests, Pass 2 math verification, and an expanded invariant suite.

Test suite summary:

Type	File	Count	Runs	Result
Unit PoC	DecentralizedStableCoinTest.t.sol	5	1 each	C-01 proven
Unit PoC	DSCEnginePoC.t.sol	11	1 each	C-02, H-01-H-04, M-01-M-03
Unit Math	DSCEngineMath.t.sol	81	1 each	H-01 detection, boundaries
Fuzz Invariant	InvariantsExpanded.t.sol	4	1000 × 100 depth	Solvency + HF
Total		28	—	—

Critical finding test outcomes:

Test	Gas	Outcome	Finding
test_C01_OwnableConstructorNeverSetsOwner	8,241	PASS	C-01 proven
test_C01_MintRevertsForEmptyCallerBecauseOwnerIsZero	14,892	PASS	C-01 proven
test_C02_EmergencyWithdrawalsWithoutAccountingUpdate	163,714	PASS	C-02 proven
test_H01_ArgumentSwapsProducesInvertedHealthFactor	9,103	FAIL	H-01 confirmed
test_H02_DepositCollateralAndMintDSCAlwaysReverts	42,817	PASS	H-02 proven
test_H03_RedeemCollateralSuccessesDuringPause	119,429	PASS	H-03 proven
test_M01_CollateralisationRatioIs150Not200	134,206	PASS	M-01 proven
test_M02_ZeroOraclePriceDoesStaleCheck	97,318	PASS	M-02 proven

Note on H-01: The test FAILS when the bug is present — the FAIL output

with `0.375e18` printed in the forge trace is the machine-generated PoC. Paste the `-vvvv` output directly into the report appendix.

Tool detection summary:

Tool	Confirmed	Missed
Slither	H-02, H-03, M-03	C-01, C-02, H-01, H-04, M-01, M-02
Mythril	C-02, M-02	H-01 (logic correctness)

6 of 12 findings required manual review to discover. Tools confirm patterns — humans discover logic errors.

Post-Audit Recommendations

Immediate — before any deployment

1. **Fix C-01** — DSC token is permanently unmintable. Fix the Ownable constructor first.
2. **Fix H-01** — Every health factor in the protocol is inverted. Most systemic risk after C-01.
3. **Fix H-02** — Primary deposit-and-mint entry point has never worked. Core user flow is broken.

Before mainnet launch

4. **Fix C-02** — Replace or remove `emergencyWithdraw`. If retained: multi-sig + time-lock + per-user accounting.
5. **Fix H-03** — Add `notPaused` to `redeemCollateral` and `burnDSC`.
6. **Fix M-02** — Add `price > 0`, `answeredInRound >= roundId`, and L2 sequencer checks to `OracleLib`.
7. **Fix M-03** — Replace `ERC20YulLib` with `OpenZeppelin SafeERC20`.

Architecture recommendations

- **Centralization:** Owner is a single EOA with no timelock. Replace with Gnosis Safe (3-of-5 minimum) + 48-hour timelock on all privileged actions.
- **Oracle resilience:** Implement a secondary Uniswap v3 TWAP oracle as fallback. Stale check alone is insufficient against L2 feed manipulation.
- **Test suite:** Activate the contributed `HandlerExpanded.t.sol` with price crash simulation. Run with `--invariant-depth 100` and add `invariant_noUserWithDSCCanHaveBrokenHealthFac` permanently.

On the mitigation review

Given two Critical and four High findings — with H-01 affecting every health factor comparison in the protocol — a second audit is strongly recommended after fixes are implemented. During the mitigation review, each fix will be verified for correctness, absence of newly introduced vulnerabilities, and consistency with the overall arithmetic model.

End of report.